Milestone 4: Data Analysis using SQL - *Amazon*

Name: *Jateen Patil*
Date: *21st Jan, 2026*

Summary: This document contains the SQL queries developed as part of the *Amazon Brazil Data Analysis* case study. Each section includes the original problem statement followed by the corresponding SQL query used to derive the required results.

The queries were executed in PostgreSQL after importing and preparing the dataset as per the given schema. These SQL snippets are provided for technical reference and traceability of the analysis.

Detailed insights, interpretations, visualizations, and business recommendations derived from these queries are presented separately in the accompanying analysis presentation.

Analysis I

1. **To simplify its financial reports, Amazon India needs to standardize payment values.** Round the average payment values to integer (no decimal) for each payment type and display the results sorted in ascending order.

```
-- Analysis I
-- 1.1 To simplify its financial reports, Amazon India needs to standardize payment values.
select payment_type as Payment_Type,round(avg(payment_value)) as Avg_Payment_Value
from payments
group by payment_type
order by Avg_payment_value asc
;
```

2. **To refine its payment strategy, Amazon India wants to know the distribution of orders by payment type.** Calculate the percentage of total orders for each payment type, rounded to one decimal place, and display them in descending order.

```
-- 1.2 To refine its payment strategy, Amazon India wants to know the distribution of orders by payment type.
select payment_type,
round(count(distinct order_id)*100.0/(select count(distinct order_id) from payments),1) as percentage_orders
from payments
group by payment_type
order by percentage_orders desc
;
```

3. **Amazon India seeks to create targeted promotions for products within specific price ranges.** Identify all products priced between 100 and 500 BRL that contain the word 'Smart' in their name. Display these products, sorted by price in descending order.

```
-- 1.3 Amazon India seeks to create targeted promotions for products within specific price ranges.
select OI.product_id,OI.price
from product as P
inner join order_items OI
on OI.product_id = P.product_id
where OI.price between 100 and 500
and P.product_category_name ilike '%smart%'
order by OI.price desc
;
```

4. **To identify seasonal sales patterns, Amazon India needs to focus on the most successful months.** Determine the top 3 months with the highest total sales value, rounded to the nearest integer.

```
-- 1.4 To identify seasonal sales patterns, Amazon India needs to focus on the most successful months.
select to_char(O.order_purchase_timestamp,'MM') as month,round(sum(OI.price+OI.freight_value),0) as total_sales
from order_items as OI
inner join orders as O
on OI.order_id = O.order_id
group by month
order by total_sales desc
limit 3
;
```

5. **Amazon India is interested in product categories with significant price variations.** Find categories where the difference between the maximum and minimum product prices is greater than 500 BRL.

```
-- 1.5 Amazon India is interested in product categories with significant price variations.
select p.product_category_name,max(o.price)-min(o.price) as price_difference
from product p
inner join order_items o
on p.product_id = o.product_id
where p.product_category_name is not null
group by p.product_category_name
having max(o.price)-min(o.price)>500
```

6. **To enhance the customer experience, Amazon India wants to find which payment types have the most consistent transaction amounts.** Identify the payment types with the least variance in transaction amounts, sorting by the smallest standard deviation first.

```
-- 1.6 To enhance the customer experience, Amazon India wants to find which payment types have the most consistent transaction amounts.
select payment_type,stddev(payment_value)  as std_deviation
from payments
where payment_type != 'not_defined'
group by payment_type
order by std_deviation asc
;
```

7. **Amazon India wants to identify products that may have incomplete name in order to fix it from their end**. Retrieve the list of products where the product category name is missing or contains only a single character.

```
-- 1.7 Amazon India wants to identify products that may have incomplete name in order to fix it from their end.
select product_id,product_category_name
from product
where product_category_name is null or length(trim(product_category_name))=1
;
```

Analysis II

1. **Amazon India wants to understand which payment types are most popular across different order value segments (e.g., low, medium, high).** Segment order values into three ranges: orders less than 200 BRL, between 200 and 1000 BRL, and over 1000 BRL. Calculate the count of each payment type within these ranges and display the results in descending order of count.

```
-- 2.   Analysis II
-- 2.1  Amazon India wants to understand which payment types are most popular across different order value segments
--      (e.g., low, medium, high).
with order_cte as (
select oi.order_id,p.payment_type,sum(oi.price+oi.freight_value) as total_order_value
from payments p
inner join order_items oi
on p.order_id=oi.order_id
group by oi.order_id,p.payment_type
)
select payment_type,count(*) as count,
case
    when total_order_value<200 then 'low'
    when total_order_value between 200 and 1000 then 'Medium'
    when total_order_value>1000 then 'High'
end as order_value_segment
from order_cte
group by order_value_segment,payment_type
order by count desc
;
```

2. **Amazon India wants to analyse the price range and average price for each product category.** Calculate the minimum, maximum, and average price for each category, and list them in descending order by the average price.

```
-- 2.2 Amazon India wants to analyse the price range and average price for each product category.
select product_category_name, min_price, max_price, avg_price
from (
select p.product_category_name,
round(avg(oi.price+oi.freight_value),2) as avg_price,
max(oi.price+oi.freight_value) as max_price,
min(oi.price+oi.freight_value) as min_price
from product p
inner join order_items oi
on p.product_id = oi.product_id
group by product_category_name
)
order by avg_price desc
;
```

3. **Amazon India wants to identify the customers who have placed multiple orders over time.** Find all customers with more than one order, and display their customer unique IDs along with the total number of orders they have placed.

```
-- 2.3 Amazon India wants to identify the customers who have placed multiple orders over time.
select c.customer_unique_id,count(c.customer_unique_id) as total_orders
from customers c
inner join orders o
on c.customer_id = o.customer_id
where order_status = 'delivered'
group by customer_unique_id
having count(customer_unique_id)>1
;
```

4. **Amazon India wants to categorize customers into different types ('New – order qty. = 1' ; 'Returning' –order qty. 2 to 4; 'Loyal' – order qty. >4) based on their purchase history.** Use a temporary table to define these categories and join it with the customers table to update and display the customer types.

```
-- 2.4 Amazon India wants to categorize customers into different types
-- ('New - order qty. = 1' ;  'Returning' -order qty. 2 to 4;  'Loyal' - order qty. >4)
create temp table if not exists temp_table(
    customer_unique_id varchar,
    customer_type varchar
)
;
insert into temp_table(
customer_unique_id,customer_type
)
select c.customer_unique_id,
case
    when count(o.order_id)=1 then 'New'
    when count(o.order_id) between 2 and 4 then 'Returning'
    when count(o.order_id)>4 then 'Loyal'
end as customer_type
from customers c
inner join orders o
on c.customer_id = o.customer_id
where order_status = 'delivered'
group by customer_unique_id
;
select *
from temp_table
;
```

5. **Amazon India wants to know which product categories generate the most revenue.** Use joins between the tables to calculate the total revenue for each product category. Display the top 5 categories.

```
--2.5 Amazon India wants to know which product categories generate the most revenue.
select
p.product_category_name,sum(oi.price+freight_value) as total_revenue
from product p
inner join order_items oi
on p.product_id = oi.product_id
group by p.product_category_name
order by total_revenue desc
limit 5
;
```

Analysis III

1. **The marketing team wants to compare the total sales between different seasons.** Use a subquery to calculate total sales for each season (Spring, Summer, Autumn, Winter) based on order purchase dates, and display the results. Spring is in the months of March, April and May. Summer is from June to August and Autumn is between September and November and rest months are Winter.

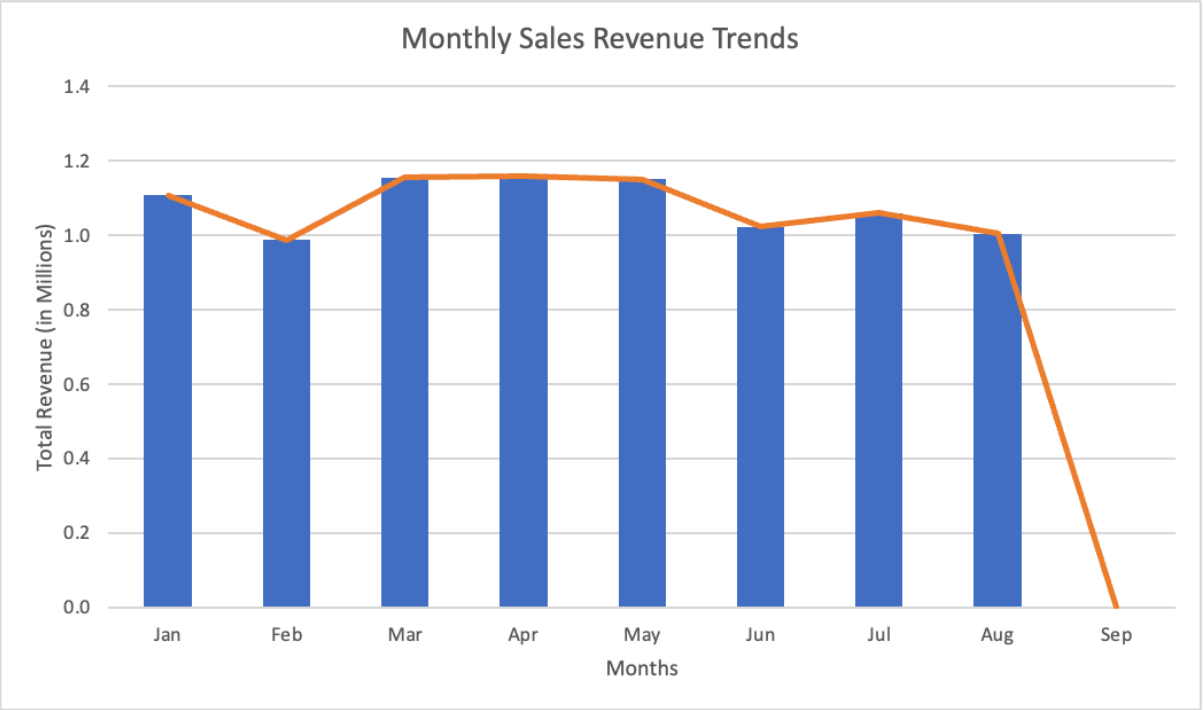
```
-- Analysis III
-- 3.1 The marketing team wants to compare the total sales between different seasons.
select season,total_sales
from (
select sum(oi.price+oi.freight_value) as total_sales,
case
    when extract(month from o.order_purchase_timestamp) in (3,4,5) then 'Spring'
    when extract(month from o.order_purchase_timestamp) in (6,7,8) then 'Summer'
    when extract(month from o.order_purchase_timestamp) in (9,10,11) then 'Autumn'
    else 'Winter'
end as season
from orders o
inner join order_items oi
on o.order_id = oi.order_id
group by season
)
order by total_sales desc
;
```

2. **The inventory team is interested in identifying products that have sales volumes above the overall average.** Write a query that uses a subquery to filter products with a total quantity sold above the average quantity.

```
-- 3.2 The inventory team is interested in identifying products that have sales volumes above the overall average.
select product_id,
total_sales
from (
select product_id,count(*) as total_sales
from order_items
group by product_id
order by total_sales desc
)
where total_sales>(
select avg(total_sales)
from (
select product_id,count(*) as total_sales
from order_items
group by product_id
)
)
;
```

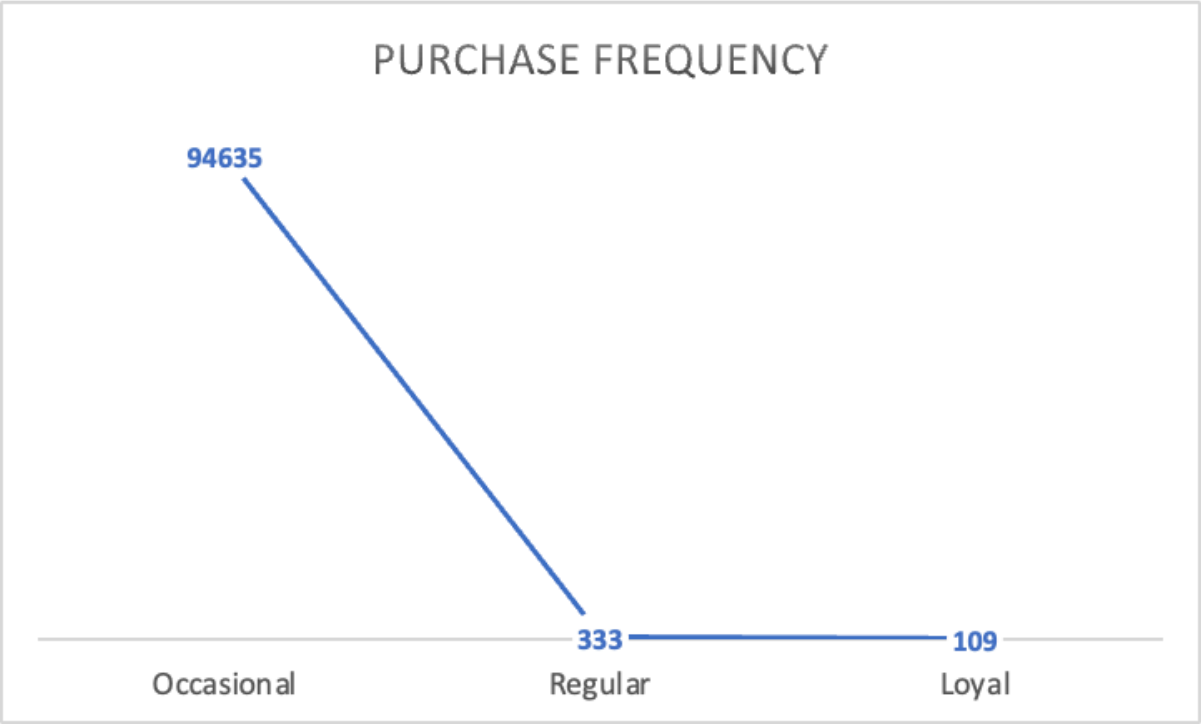
3. **To understand seasonal sales patterns, the finance team is analysing the monthly revenue trends over the past year (year 2018).** Run a query to calculate total revenue generated each month and identify periods of peak and low sales. Export the data to Excel and create a graph to visually represent revenue changes across the months.

```
-- 3.3  To understand seasonal sales patterns,
--      the finance team is analysing the monthly revenue trends over the past year (year 2018).
select extract(month from o.order_purchase_timestamp) as month,sum(oi.price+oi.freight_value) as total_revenue
from order_items oi
inner join orders o
on oi.order_id = o.order_id
where extract(year from o.order_purchase_timestamp) = 2018
group by month
;
```



4. **A loyalty program is being designed for Amazon India.** Create a segmentation based on purchase frequency: ‘Occasional’ for customers with 1-2 orders, ‘Regular’ for 3-5 orders, and ‘Loyal’ for more than 5 orders. Use a CTE to classify customers and their count and generate a chart in Excel to show the proportion of each segment.

```
-- 3.4 A loyalty program is being designed  for Amazon India.
with order_cte as (
select c.customer_unique_id as customer_unique_id,count(distinct oi.order_id) as order_count
from customers c
inner join orders oi
on c.customer_id = oi.customer_id
group by c.customer_unique_id
),
customer_segment_cte as (
    select customer_unique_id,
    case
        when order_count<=2 then 'Occasional'
        when order_count between 3 and 5  then 'Regular'
        else 'Loyal'
    end as customer_type
    from order_cte
)
select customer_type,count(*) as customer_count
from customer_segment_cte
group by customer_type
;
```



5. **Amazon wants to identify high-value customers to target for an exclusive rewards program.** You are required to rank customers based on their average order value (avg_order_value) to find the top 20 customers.

```
-- 3.5 Amazon wants to identify high-value customers to target for an exclusive rewards program.
with mycte as (
select customer_id,round(total_value,2) as avg_order_value,
dense_rank() over(order by round(total_value,2) desc) as customer_rank
from (
select c.customer_id as customer_id,sum(oi.price+oi.freight_value)/count(distinct oi.order_id) as total_value
from order_items oi
inner join orders o
on o.order_id = oi.order_id
inner join customers c
on c.customer_id = o.customer_id
group by c.customer_id
)
)
select customer_id,avg_order_value,customer_rank
from mycte
where customer_rank<=20
;
```

7.**To understand how different payment methods affect monthly sales growth, Amazon wants to compute the total sales for each payment method and calculate the month-over-month growth rate for the past year (year 2018).** Write query to first calculate total

monthly sales for each payment method, then compute the percentage change from the previous month.

```sql
-- 3.7  To understand how different payment methods affect monthly sales growth,
--      Amazon wants to compute the total sales for each payment method and calculate the month-over-month growth rate for the past year (year 2018).
WITH order_value AS (
    SELECT
        o.order_id,
        extract(month from  o.order_purchase_timestamp) AS sales_month,
        SUM(oi.price + oi.freight_value) AS order_total
    FROM orders o
    JOIN order_items oi
        ON o.order_id = oi.order_id
    WHERE EXTRACT(YEAR FROM o.order_purchase_timestamp) = 2018
    GROUP BY o.order_id, sales_month
),
monthly_payment_sales AS (
    SELECT
        p.payment_type,
        ov.sales_month,
        SUM(ov.order_total) AS monthly_sales
    FROM order_value ov
    JOIN payments p
        ON ov.order_id = p.order_id
    GROUP BY p.payment_type, ov.sales_month
),
mom_growth AS (
    SELECT
        payment_type,
        sales_month,
        monthly_sales,
        LAG(monthly_sales) OVER (
            PARTITION BY payment_type
            ORDER BY sales_month
        ) AS prev_month_sales
    FROM monthly_payment_sales
)
SELECT
    payment_type,
    sales_month,
    monthly_sales,
    ROUND(
        (monthly_sales - prev_month_sales) * 100.0 / prev_month_sales,
        2
    ) AS mom_growth_percentage
FROM mom_growth
ORDER BY payment_type, sales_month;
;
```